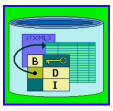


Structured Query Language (SQL 2)

Segunda Parte

2.4. Recupero de Tuplas

SQL no es un lenguaje que respete todas las restricciones del modelo relacional. De hecho, las tablas ***NO son un conjunto de tuplas*** ya que aceptan elementos repetidos. Se las considera ***multiconjuntos o bolsas de tuplas (multiset o bags)***. **P**odemos crear una tabla sin definir una clave, y por lo tanto, en ese caso, la restricción de clave no se garantizará.

A diferencia de los lenguajes de consulta Álgebra Relacional y Cálculo de Predicados, en general, una operación SQL no eliminará repeticiones, salvo que se lo solicite explícitamente.

El hecho de que SQL no elimine repeticiones puede tener alguna ventaja: no pierde tiempo eliminando las duplicaciones y es especialmente útil cuando se usan funciones de agregación, como veremos más adelante, cosa que el álgebra relacional o cálculo de predicados no existe. La desventaja está en el dolor de cabeza que puede traer a los programadores al ver algún reporte con filas repitiendo los mismos valores.

SQL sólo garantiza la eliminación de duplicaciones cuando se utilizan operaciones conjuntistas: **UNION, INTERSECT, EXCEPT**. Más aún, si en esos casos nos interesara mantener las duplicaciones tendríamos que hacerlo explícitamente con la cláusula **ALL**.

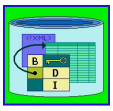
La forma de expresar una consulta consiste en distintas cláusulas, donde sólo las dos primeras son obligatorias, pero de colocar alguna más deben aparecer en el orden indicado:

Sintaxis *(*Las cláusulas SELECT pueden anidarse)

```
SELECT [DISTINCT] atrib1 [AS otroNombre1],..., atribN [AS otroNombreN]
FROM     lista_de_tablas
[ WHERE condicion ]
[
   GROUP BY lista_atributos_que_agrupan
   [ HAVING condicion_para_el_grupo]
]
[ ORDER BY lista_atributos ]
```

A continuación, analizaremos a qué corresponde cada cláusula.

Silvia Alicia Gómez & Leticia Irene Gómez



❖ **FROM**

Indica todos los nombres de tablas que intervienen en el procesamiento de la consulta. En la lista_de_tablas se puede dar un "alias" o variable de correlación para cada una de ellas. Esto se hace acompañando el nombre de la tabla por AS vble. Usar AS para dar un alias a cierta tabla corresponde al operador de renombramiento (**ρ**) del Álgebra Relacional.

Las tablas usadas en esta cláusula pueden ser persistentes o temporarias (generadas a través de un SELECT ANIDADO, en cuyo caso es obligatorio que la tabla generada reciba un nombre).

Aclaración

El ANSI exige usar la cláusula AS para expresar renombramiento.

PosgreSQL acepta expresar renombramiento y permite usar/omitir la cláusula AS.

❖ **WHERE**

Indica la condición que se coloca en el **operador de selección** del Álgebra Relacional **(σ).** Permite expresar la condición que, de evaluarse VERDADERA, recuperará las tuplas en el resultado.

❖ **GROUP**

Permite armar subgrupos de tuplas. No tiene equivalente en el Álgebra Relacional. Lo veremos en la sección 2.2.4.

❖ **HAVING**

Permite evaluar condiciones sobre groupos (GROUP BY). No tiene equivalente en el Álgebra Relacional. Lo veremos en la sección 2.2.4.

❖ **ORDER BY**

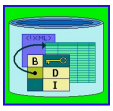
Permite indicar el orden de las tuplas que aparecerán en el resultado. Cada uno de los atributos en esta lista puede estar acompañado por la palabra ASC o DESC según se desee que dicho atributo se ordene ascendentemente o descendentemente. De omitir dichas palabras, se asume ASC. Como ya sabemos, no hay operador semejante en el Álgebra Relacional.

❖ **SELECT**

Corresponde al operador de proyección del Álgebra Relacional (**π**). Indica la lista de atributos que se recuperarán en la consulta. Cada uno de ellos puede estar acompañado por un AS que permite que el nombre del atributo cambie en la relación resultante. Se puede utilizar el símbolo * si se quieren todos los atributos en la relación resultante. SQL no filtra repetidos, por lo tanto, si se deseara eliminar duplicaciones se debería utilizar la palabra DISTINCT (un solo distinct porque se aplica a todo el resultado y no sobre cada atributo!)

Usar AS para dar un alias a cierto atributo corresponde al operador de renombramiento (**ρ**) del Álgebra Relacional. Esta es la última cláusula que ejecuta el DBMS por lo que un atributo renombrado con AS aquí no puede usar su nuevo nombre en el resto de las cláusulas.

Silvia Alicia Gómez & Leticia Irene Gómez



(PostgreSQL no respeta el ANSI exactamente al respecto)

2.4.1 Ambigüedades y sus Soluciones

Vamos ahora a considerar los problemas de ambigüedad que pueden surgir en la expresión de las consultas y cómo se solucionan en SQL. Las ambigüedades pueden ocurrir:

- Cuando se usa más de una tabla (en el FROM) y ambas tienen atributos con el mismo nombre los cuales se usan en alguna expresión. Así es como tenemos que recurrir a una forma de identificar a cuál de los atributos nos estamos refiriendo. SQL exige, en ese caso, agregar al atributo repetido el prefijo del nombre de la tabla a la cual pertenece. Obviamente también podemos prefijar a un atributo aunque no se repita.

  Ejemplo:

```
SELECT editorial.codEdit, nombre
FROM editorial, libro
WHERE  editorial.codEdit = libro.codEdit
```

- Cuando la misma tabla aparece más de una vez (en el FROM) el esquema anterior no sirve porque por más que usemos un prefijo, el mismo estaría repetido otra vez. En ese caso SQL exige dar un ***alias o definir una variable de correlación*** para cada tabla que aparezca repetido en la cláusula FROM. Esta forma de representar a una tupla de la tabla por un nombre (y no cada uno de sus atributos) muestra la influencia que ejerció el Cálculo Relacional de tuplas sobre este lenguaje.

  Ejemplo:

```
SELECT nombre
FROM   libro AS libro1, libro AS libro2, editorial
WHERE libro1.codEdit= libro2.codEdit
         AND libro1.ISBN <> libro2.ISBN
         AND libro1.codEdit = editorial.codEdit
```

2.4.2 Cláusula FROM

2.4.2.1 Uso de Juntas

SQL-2 permite utilizar en la cláusula FROM las juntas discutidas en los otros lenguajes de consulta.

❖ **Producto Cartesiano**

En el Álgebra Relacional tenemos el operador **producto cartesiano (X)**. En SQL se denomina CROSS JOIN.

Sin embargo, es posible directamente calcular en la cláusula FROM una lista de tablas, y eso se interpreta como producto cartesiano entre ellas.

Silvia Alicia Gómez & Leticia Irene Gómez

```
r CROSS JOIN s

o bien

(r CROSS JOIN s) AS nueva_relacion(lista_atributos_renombrada)

O bien

r, s
```

- **Tetha Join** (recordar que las tuplas que no machean no aparecen en el resultado)

  En el Álgebra Relacional el "join a secas" es el **natural join**, en cambio en SQL es el **theta join** y debe especificarse condición (y obviamente repite las columnas de junta para cada una de las relaciones intervinientes). La palabra INNER no hace falta especificarla:

```
r [INNER] JOIN s ON condicion

o bien

(r [INNER] JOIN s ON condición) AS
nueva_relacion(lista_atributos_renombrada)
```

- **Junta Natural**.

  Recordar que las tuplas que no machean no aparecen en el resultado. No hace falta indicar las condiciones de junta porque son la igualdad de nombre de atributos, y las columnas repetidas aparecen una sola vez. Si no hay atributos en común,entonces es un producto cartesiano.

```
r NATURAL JOIN s

o bien

(r NATURAL JOIN s) AS nueva_relacion(lista_atributos_renombrada)
```

Silvia Alicia Gómez & Leticia Irene Gómez

Aclaración (Suponiendo que la tabla ARTICULO y PROVEE juntan por la columna CODIGO)

- En los **DBMS** donde ni siquiera exista el JOIN, hay que reformularlo indicando las condiciones de junta en el WHERE y filtrando las columnas deseadas:

```
SELECT  descripcion, nombre, articulo.codigo
FROM  articulo, provee
WHERE articulo.codigo = provee.codigo
```

❖ **Semijunta natural o Outer Join**.

Las tuplas que no coinciden con ninguna tupla de la otra relación aparecen en el resultado con NULL donde no se conoce el valor. Hay distintos tipos, según cuál de las relaciones quiere conservar las tuplas, aunque no encuentren coincidencia. En SQL hay que especificar la condición de junta por medio de la subcláusula JOIN ON.

Si sólo es importante conservar las tuplas de r que no encuentren correspondencia con s, se dice que r es dominante o preservante. Obviamente la operación es asimétrica:

```
r   LEFT OUTER JOIN    s   ON   condición

o bien

(r       LEFT   OUTER   JOIN         s       ON       condición)     AS
nueva_relacion(lista_atributos_renombrada)
```

Si sólo es importante conservar las tuplas de s que no encuentren correspondencia con r, se dice que s es dominante o preservante. Obviamente la operación es asimétrica:

```
r   RIGHT OUTER JOIN    s   ON   condición

o bien

(r       RIGHT   OUTER   JOIN         s       ON       condición)     AS
nueva_relacion(lista_atributos_renombrada)
```

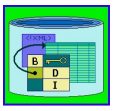
Si se conservan las tuplas de las dos relaciones, aunque no tengan correspondencia:

```
r   FULL OUTER JOIN    s   ON  condicion

o bien

(r        FULL   OUTER   JOIN          s        ON      condicion)   AS
nueva_relacion(lista_atributos_renombrada)
```

Silvia Alicia Gómez & Leticia Irene Gómez



2.4.2.2 Consultas anidadas (Nested Query)

Una consulta anidada es una consulta que contiene a otra. La más interna se denomina subconsulta (*subquery*). En la cláusula FROM no sólo pueden participar tablas persistentes sino también tablas temporarias generadas a través de subconsultas. Es obligatorio que la tabla generada reciba un nombre y obviamente sólo puede referenciarse dentro de la misma consulta.

Ejemplo:

```
SELECT ISBN, precioMod
FROM
     (SELECT  ISBN, precio * 0.20
     FROM libro
     WHERE precio > 500) AS L1(ISBN, precioMod)
```

Ejemplo:

```
SELECT ISBN, precioMod
FROM
     (SELECT  ISBN, precio * 0.20, codEdit
     FROM libro
     WHERE precio > 500) AS L1(ISBN, precioMod, codEdit),
     editorial
WHERE  editorial.codEdit = L1.codEdit
```

2.4.3 Expresiones en la Cláusula WHERE

En esta cláusula hay que armar expresiones que puedan satisfacerse (evaluarse como verdaderas o falsas). Para ello disponemos de distintos operadores.

- Por medio de los operadores lógicos **AND, NOT** y **OR** podemos armar expresiones. Se pueden usar paréntesis.
  Conviene usar siempre paréntesis explícitamente para evitar sorpresas al mezclar los operadores en una misma expresión.

- Contamos con los operadores **<, <=, >, >=, <>** para comparar valores de atributos entre sí o con constantes.

  Ejemplo:

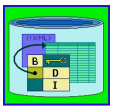
```
SELECT ISBN, precio
FROM libro
WHERE precio > 100 AND precio < 2000
```

- **Comparación de NULL:** El valor NULL se utiliza para indicar cualquiera de los siguientes tres estados: no aplicable, aplicable pero desconocido o no se sabe si es aplicable. Como no se conoce precisamente qué se está indicando con un valor ***null***, SQL no permite comparar los valores nulos entre sí, o más precisamente "WHERE atr = NULL" o "WHERE atr <> NULL" se evalúa como falso. Para poder saber si un atributo es o no nulo hay que hacerlo explícitamente con el predicado **IS NULL** o bien **IS NOT NULL**, según corresponda.

  Ejemplo:

```
SELECT ISBN
FROM libro
WHERE precio IS NOT NULL
```

Silvia Alicia Gómez & Leticia Irene Gómez



- Para los string se puede utilizar el operador **LIKE** que permite buscar apariciones de patrones dentro de otros. Se lo usa en conjunción con los comodines % y _. El primero matchea con cualquier substring y el segundo con un solo caracter. El lugar donde se coloca el comodín es muy importante. Por ejemplo, para recuperar los empleados cuyo apellido contiene PEREZ podría usarse **... nombre LIKE '%Perez%'**

  Ejemplo:

```
SELECT ISBN, precio
FROM libro
WHERE descrip LIKE 'Introduccion %'
```

  - Disponemos del operador **IN**, o sus sinónimos **=ANY** o bien **=SOME** que permite averiguar si el valor de un atributo se encuentra entre alguno de los valores de cierta lista de valores. Más aún, dicha lista puede ser fija o bien puede surgir del resultado de otra consulta (aquí también se pueden utilizar SELECT anidados).

  PostgreSQL no permite usar listas fijas para ANY ni para SOME, pero sí para IN. Tener en cuenta que un atributo no puede ser comparado directamente contra un conjunto de valores, por eso es importante este tipo de operadores.

  Ejemplo:

```
SELECT ISBN, precio
FROM libro
WHERE precio IN (10, 20, 30)
```

  Ejemplo:

```
SELECT ISBN, precio
FROM libro
WHERE codEdit IN (SELECT codEdit FROM editorial)
```

También puede utilizarse para preguntar si los valores de k-atributos se encuentra entre alguno de los valores de una lista también formada por k-atributos. En este caso averiguamos si los 2 atributos precio y editorial se encuentran en la lista indicada. Debe haber compatibilidad en número de atributos y valores proporcionados.

Ejemplo:

```
SELECT *
FROM libro
WHERE (precio, editorial) IN
   ( (100, 'PH '), (300, 'AW ' ), (500, 'AW ' ))
```

- También contamos con los operadores **<ANY, <=ANY, >ANY, >=ANY, <>ANY**, o bien sus sinónimos respectivos **<SOME, <=SOME, >SOME, >=SOME, <>SOME** que se usan para comparar un valor de atributo con una lista de valores. Dicha lista también puede ser fija o bien puede surgir del resultado de otra consulta (aquí también se pueden utilizar SELECT anidados).
  PostgreSQL no permite usar listas fijas para ANY ni SOME. Tener en cuenta que un atributo no puede ser comparado directamente contra un conjunto de valores, por eso es importante este tipo de operadores.

  Ejemplo:

```
SELECT ISBN, precio
FROM libro
WHERE codEdit <ANY (SELECT codEdit FROM editorial)
```

Silvia Alicia Gómez & Leticia Irene Gómez

- Contamos con los operadores **=ALL, <ALL, <=ALL, >ALL, >=ALL, <>ALL** que permite comparar el valor de un atributo contra todos los valores de la lista de valores indicada. Si verifica la condición pedida contra todos los elementos de la lista devuelve TRUE, sino FALSE. Dicha lista también puede ser fija o bien puede surgir del resultado de otra consulta (aquí también se pueden utilizar SELECT anidados).

  PostgreSQL no permite usar listas fijas para ALL. Tener en cuenta que un atributo no puede ser comparado directamente contra un conjunto de valores, por eso es importante este tipo de operadores.

- El operador **EXISTS** se usa con SELECT anidados. Chequea cada tupla candidata del SELECT externo respecto del resultado de la consulta anidada (interno). Si el resultado de evaluar la consulta anidada es distinto de vacío (hay por lo menos una tupla), entonces la tupla exterior aparece en la respuesta, caso contrario no. Como se observa, la idea es que la consulta anidada exprese en su condición alguna comparación con algún atributo de la tupla del SELECT externo. O sea, se satisface para cada tupla del SELECT externo si el conjunto resultante del SELECT interno es no vacío, y en caso de que así sea la tupla aparece en la respuesta. Esto significa que "EXISTS (relación)" es verdadera si y sólo sí la relación es no vacía". Cuando la consulta anidada usa atributos de la consulta externa se dice que ambas consultas están en ***correlación (correlated queries).***

  Ejemplo:

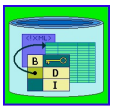
```
SELECT *
FROM libro
WHERE EXISTS (SELECT *
              FROM editorial
              WHERE editorial.codEdit = libro.codEdit)
```

- El operador **NOT EXISTS** se usa con SELECT anidados. Chequea cada tupla candidata del SELECT externo respecto del resultado de la consulta anidada (interno). Si el resultado de evaluar la consulta anidada es vacío (no hay tupla), entonces la tupla exterior aparece en la respuesta, caso contrario no. Como se observa, la idea es que la consulta anidada exprese en su condición alguna comparación con algún atributo de la tupla del SELECT externo. O sea, se satisface para cada tupla del SELECT externo si el conjunto resultante del SELECT interno es vacío, y en caso de que así sea la tupla aparece en la respuesta. Esto significa que "NOT EXISTS (relación)" es verdadera si y sólo sí la relación es vacía". Cuando la consulta anidada usa atributos de la consulta externa se dice que ambas consultas están en ***correlación (correlated queries).***

  Ejemplo:

```
SELECT *
FROM libro
WHERE NOT EXISTS (SELECT *
                  FROM editorial
                  WHERE editorial.codEdit = libro.codEdit)
```

Silvia Alicia Gómez & Leticia Irene Gómez



Aclaración para Consultas Anidadas

En los lenguajes de programación las variables pueden redefinirse en bloques más anidados, y las variables más internas tienen prioridad sobre las otras. Algunos lenguajes como C++ permiten que las variables de más afuera puedan seguir referenciándose por medio de alguna forma sintáctica que permita explícitamente indicar el alcance.

En SQL cuando se tienen SELECT anidados el más interno es el que tiene prioridad, pero si se quiere referenciar algún atributo más externo se recurre al método de agregar al atributo el prefijo que indique el nombre de la relación a la que pertenece. El ejemplo anterior podría escribirse como:

```
SELECT *
FROM libro
WHERE NOT EXISTS (SELECT *
                  FROM editorial
                  WHERE codEdit = libro.codEdit)
```

2.4.4 Funciones de Agregación para las Cláusulas SELECT y HAVING

En vez de recuperar los valores de los atributos también se les puede aplicar ciertas funciones, llamadas de agregación o sumarización. Precisamente se denominan funciones de sumarización porque reemplazan un grupo de valores por uno solo de ellos. Dichas funciones pueden ser:

Nombre	Descripción
COUNT	Devuelve la dimensión (cantidad de valores) del conjunto o multi-conjunto. Si se indica COUNT(atributo) se calcula sobre el multi-conjunto. Si se indica COUNT(DISTINCT atributo) se calcula sobre el conjunto (o sea, primero se eliminan repetidos) La expresión COUNT(*) cuenta la cantidad de tuplas.
SUM	Calcula la suma de los valores del atributo pedido en el conjunto o multi-conjunto. Si se indica SUM(atributo) se calcula sobre el multi-conjunto. Si se indica SUM(DISTINCT atributo) se calcula sobre el conjunto (o sea, primero se eliminan repetidos)
AVG	Calcula el promedio de los valores del atributo pedido en el conjunto o multiconjunto. Si se indica AVG(atributo) se calcula sobre el multi-conjunto. Si se indica AVG(DISTINCT atributo) se calcula sobre el conjunto (o sea, primero se eliminan repetidos)
MAX	Calcula el máximo valor del atributo pedido. Obviamente no tiene sentido diferenciar entre conjunto o multi-conjunto porque el máximo es único. Por lo tanto NO se puede indicar DISTINCT.
MIN	Calcula el mínimo valor del atributo pedido. Obviamente no tiene sentido diferenciar entre conjunto o multi-conjunto porque el mínimo es único. Por lo tanto NO se puede indicar DISTINCT.

Silvia Alicia Gómez & Leticia Irene Gómez

Restricciones en el uso de una Función de Agregación

Estas funciones sólo pueden aparecer en la cláusula HAVING o SELECT.

Si se usa una función de agregación en alguna de estas cláusulas, entonces todo atributo que aparezca en dicha cláusula debe, o bien estar agregado, o bien participar necesariamente de la cláusula GROUP BY.

Existe otra restricción para el uso en la cláusula HAVING: debe usarse necesariamente junto con la cláusula GROUP BY (ver próximo ítem).

Muy Importante

- Las funciones de agregación no pueden aplicarse sucesivamente. Ejemplo, es inválido escribir SUM(AVG(sueldo))

- Para todas las funciones de agregación, a excepción de COUNT, se define el siguiente comportamiento: Antes de aplicar la función de agregación se descartan todos los valores NULL del conjunto. Si luego de esto el conjunto quedara vacío se devuelve una única tupla con valor NULL.

- Para la función de agregación COUNT se define el siguiente comportamiento:
  a) Si se aplica a una columna COUNT(nombreCol) se descartan todos los valores NULL del conjunto antes de aplicarla, y si luego de eso el conjunto quedara vacío se devuelve una única tupla con valor 0.
  b) Si en cambio se aplica como COUNT(*) no se descartan las tuplas con valores NULL al realizar la evaluación.

- La aplicación de una función de agregación siempre devuelve una tupla.

Algunos ejemplos:

Supongamos que se tiene la tabla EMPLEADO

EMPLEADO		
Nombre	**Sueldo**	**Edad**
'Ana'	500	23
'Juan	1200	34
'Clara'	2400	30
'Jose'	3000	34
'Maria'	1000	Null
'Ema'	2400	null

Si aparece una función de agregación en la cláusula SELECT pero NO hay cláusula GROUP BY, entonces significa que se aplica la función solicitada sobre el atributo indicado en el total de tuplas considerada como conjunto o multi-conjunto (según se use o no DISTINCT dentro del argumento de la función).

Silvia Alicia Gómez & Leticia Irene Gómez

Ejemplo 1:

Con esta consulta se calcula, sobre todos los empleados, la cantidad de sueldos que se tienen. El cálculo se realiza sobre el multi-conjunto (500, 1200, 2400, 3000, 1000, 2400). El resultado obtenido será **6**

```
SELECT COUNT(Sueldo)
FROM empleado;
```

Ejemplo 2:

Con esta consulta se calcula, sobre todos los empleados, la cantidad de sueldos distintos que se tienen. El cálculo se realiza sobre el conjunto (500, 1200, 2400, 3000, 1000). El resultado obtenido será **5**

```
SELECT COUNT(DISTINCT sueldo)
FROM empleado;
```

Ejemplo 3:

Con esta consulta se calcula, sobre todos los empleados, la cantidad de edades que se tienen. El cálculo se realiza sobre el multi-conjunto (23, 34, 30, 34). Notar que los valores NULL no se tienen en cuenta para el cálculo. El resultado obtenido será **4**

```
SELECT COUNT(edad)
FROM empleado;
```

Ejemplo 4:

Con esta consulta se calcula, sobre todos los empleados, la cantidad de edades distintas que se tienen. El cálculo se realiza sobre el conjunto (23, 34, 30). Notar que los valores NULL no se tienen en cuenta para el cálculo. El resultado obtenido será **3**

```
SELECT COUNT(DISTINCT edad)
FROM empleado;
```

Ejemplo 5:

Con esta consulta se calcula, sobre todos los empleados, el promedio de sus edades. El cálculo se realiza sobre el multi-conjunto (23, 34, 30, 34). Notar que los valores NULL no se tienen en cuenta para el cálculo. El resultado obtenido será **30.25**

```
SELECT AVG(edad)
FROM empleado;
```

Silvia Alicia Gómez & Leticia Irene Gómez

Ejemplo 6:

Con esta consulta se calcula, sobre todos los empleados, el promedio de las edades previa eliminación de repetidos. El cálculo se realiza sobre el conjunto (23, 34, 30). Notar que los valores NULL no se tienen en cuenta para el cálculo. El resultado obtenido será **29**

```
SELECT AVG(DISTINCT edad)
FROM empleado;
```

Ejemplo 7:

Con esta consulta se quiere calcular la máxima de las edades, acompañada por el nombre de la persona correspondiente. Sin embargo es INVALIDA!!!!

```
SELECT MAX(edad), nombre
FROM empleado;
```

Esta **sintaxis no es válida**. El motivo por el cual NO es una sentencia SQL es porque cualquiera de las funciones de agregación devuelven un UNICO valor (un única tupla) por lo tanto es IMPOSIBLE acompañar dicho valor por sus nombres correspondientes (de hecho en el ejemplo hay dos personas que tienen la edad máxima 34). Ver recuadro sobre "Restricciones en el uso de una Función de Agregación" en páginas previas.

Ejemplo 8:

¿Son siempre estas dos consultas equivalentes?

```
SELECT AVG(edad) AS prom
FROM empleado;
```

```
SELECT SUM(edad)/COUNT(*) AS prom
FROM empleado;
```

NO. En el ejemplo anterior, la primera consulta arroja 30.25, pero la segunda 121/6 o sea 20.1666. El problema es la presencia de valores NULL.

Ejemplo 9:

Sea la siguiente tabla

SOPRESA
nro
Null
Null
Null

Silvia Alicia Gómez & Leticia Irene Gómez

Al ejecutar

```
SELECT MAX(nro), MIN(nro), AVG(nro), SUM(nro), COUNT(nro), COUNT(*)
FROM sorpresa;
```

se obtendría

```
NULL   NULL   NULL   NULL   0    3
```

Notar que salvo en COUNT, todas las funciones arrojan un NULL cuando el único valor presente es NULL. En ese caso el COUNT no se comporta igual si se pregunta por COUNT(*) que por COUNT(col).

2.4.5 Cláusula Group By

En el inciso anterior se mostró el uso de funciones de agregación en la cláusula SELECT.

Hemos visto que las funciones de agregación en la cláusula SELECT no permiten colocar junto a ellas otros atributos que no estén afectados de otras funciones de agregación, a excepción de que se use la cláusula GROUP BY.

El ejemplo 7 se puso en evidencia el error conceptual de pretender obtener en una sola tupla un único atributo (obtenido por la función de agregación) acompañado quizás por varios valores.

Si en vez de obtener un único valor que sea el resultado de aplicar la función de agregación sobre todo el conjunto o multiconjunto, se armaran "subgrupos de tuplas" se podría aplicar funciones de agregación sobre cada una de ellas. Así por cada subgrupo obtendríamos una única tupla.

Armar subgrupos implica indicar qué tuplas formarían un mismo grupo. Para ello se especifica los atributos coincidentes que harán que las tuplas con valores iguales en dicho atributos aparezcan en el mismo subgrupo. Para un subgrupo en particular tenemos entonces atributos coincidentes (los que definieron el grupo) y otros que no. Para un grupo determinado los atributos coincidentes son únicos en él y por lo tanto pueden recuperarse en la cláusula SELECT. Para un grupo determinado los atributos no coincidentes forman un conjunto y por lo tanto no podrían ser recuperados en la cláusula SELECT, a menos que se les aplique una función de agregación que los transforme en únicos.

Resumiendo**, si se arman subgrupos, por cada uno de ellos, puede obtenerse una única tupla en la cláusula SELECT armada directamente por atributos que agruparon o por atributos cualesquiera con tal de que se los afecte por funciones de agregación.**

Esto es lo que comentamos como "Restricciones en el uso de una Función de Agregación" en páginas previas.

No existe un operador equivalente en el Álgebra Relacional, por eso es que SQL no tiene un poder expresivo equivalente a aquel.

Silvia Alicia Gómez & Leticia Irene Gómez

Muy Importante

Sabemos que dos valores NULLs no se pueden comparar entre sí porque no se van a considerar equivalentes (WHERE NULL = NULL o bien WHERE NULL<> NULL se evalúan como FALSE).

Sin embargo, la agrupación considera que los valores NULL son equivalentes entre sí, o sea "GROUP BY atributo" juntaría en el mismo subgrupo aquellas tuplas donde "atributo" valga NULL.

A continuación, mostraremos nuevos ejemplos sobres estos conceptos.

Ejemplo 1:

Con esta consulta se quiere calcular el sueldo promedio por edad. Notar que si no se aplicara la cláusula GROUP BY se obtendría el sueldo promedio sin importar la edad.

```
SELECT edad, AVG(sueldo)
FROM empleado
GROUP BY edad;
```

Supongamos que la tabla es la siguiente

EMPLEADO		
Nombre	**Sueldo**	**Edad**
'Ana'	500	23
'Juan'	1200	34
'Clara'	2400	30
'Jose'	3000	34
'Maria'	1000	null
'Ema'	2400	null

Entonces se obtiene:

```
   EDAD      AVG(SUELDO)
--------   -----------
     23          500
     30         2400
     34         2100
   null         1700
```

¿Cómo se explica el resultado anterior?

- *Primer Paso*

  Se agrupa por el atributo edad, formando internamente:

Silvia Alicia Gómez & Leticia Irene Gómez

Calculo auxiliar		
Edad	**Sueldo**	**Nombre**
23	500	'Ana'
30	2400	'Clara'
34	1200	'Juan'
	3000	'Jose'
null	1000	'Maria'
	2400	'Ema'

- *Segundo Paso*

Ahora que se armaron los subgrupos coincidentes en el atributo Edad, se puede recuperar en la cláusula SELECT directamente los atributos que agruparon (en el ejemplo edad) y/o cualquiera de los otros siempre y cuando se los afecte de una función de agregación. Dado que de cada subgrupo sólo debe obtenerse una tupla representante no se puede recuperar "sueldo" si no se lo afecta de una función de agregación que lo transforme en un único valor. Ídem para el campo Nombre. Para el ejemplo sólo se pedía edad acompañado del promedio de sueldos, o sea el resultado final es:

Rta	
Edad	**AVG(SUELDO)**
23	500
30	2400
34	2100
Null	1700

Nótese además que internamente los DBMS, para poder agrupar por los atributos pedidos, suelen ordenar ascendentemente por ellos (para tener consecutivos los idénticos), por lo tanto cuando se usa un GROUP BY el resultado puede resultar ordenado ascendentemente por los atributos pedidos. Igualmente podrían usar otra técnica, si hace falta ordenar explícitamente.

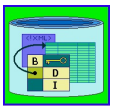
Ejemplo 2:

Vamos a explicar el resultado obtenido al ejecutar la siguiente consulta:

```
SELECT sueldo, AVG(edad)
FROM empleado
GROUP BY sueldo;
```

```
   SUELDO      AVG(EDAD)
----------     ----------
       500          23
      1000        null
      1200          34
      2400          30
      3000          34
```

Silvia Alicia Gómez & Leticia Irene Gómez



- *Primer Paso*

Se arman los grupos por sueldos equivalentes

CALCULO AUXILIAR		
Sueldo	**Edad**	**Nombre**
500	23	'Ana'
1000	null	'Maria'
1200	34	'Juan'
2400	30	'Clara'
	Null	'Ema'
3000	34	'Jose'

- *Segundo Paso*

Se devuelve una tupla por cada subconjunto con el sueldo acompañado por el AVG de las edades en su subconjunto, obviamente con las convenciones antes citadas. Por ejemplo, para sueldo 2400, el AVG se aplica sólo sobre el valor 30 porque el valor NULL no cuenta al aplicar la función de agregación. Para sueldo 1000 el AVG devuelve el valor NULL ya que al eliminar los valores nulos queda un conjunto vacío. Finalmente se obtiene:

Rta	
Sueldo	**AVG(Edad)**
500	23
1000	Null
1200	34
2400	30
3000	34

Ejemplo 3:

Si tuviéramos la tabla con las PENALIZACIONES incurridas por jugadores (TP 6A):

JUGADOR		
codigo	**nombre**	**telefono**
A02	Barros	4542-8872
B04	Taranto	4314-2345
B06	Vilar	null
R02	Sanchez	4785-6562
R07	Peirano	4612-3332
R11	Cardoso	4314-8102

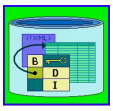
PENALIZACION		
Codigo	**fecha**	**monto**
B06	24/10/1998	120
B04	12/05/1999	120
R07	15/07/1999	100
R07	30/10/1998	120
R11	18/07/1999	150

y se quisiera obtener el promedio de los montos acumulados de los jugadores, podríamos pensar en realizar un SUM(monto) agrupado por jugador y luego el promedio de estos valores:

```
SELECT AVG(SUM(monto))
FROM penalizacion
GROUP BY codigo
```

Sin embargo, esta sintaxis es INVÁLIDA: las funciones de agregación no pueden anidarse.

Silvia Alicia Gómez & Leticia Irene Gómez



Tenemos que recurrir a generar una consulta anidada que primero calcule la suma de montos de cada jugador y luego generar el promedio para sus tuplas:

```
SELECT  AVG( acum)
FROM (SELECT SUM( monto )
       FROM penalizacion
       GROUP BY codigo) AS AUXI( acum)
```

Con lo cual se obtendría una tabla temporaria (luego de la ejecución de la consulta no puede referenciarse) con los valores (120, 120, 220, 150) y luego devolvería el promedio de sus 4 valores:

```
valor
152
```

2.4.6 Cláusula Having

En los ejemplos anteriores aplicamos las funciones de agregación sólo en la cláusula SELECT. Por supuesto que se puede utilizar filtros antes de aplicar las funciones de agregación.

Ejemplo 1:

La siguiente consulta calcula el promedio de los sueldos para aquellas personas que tengan más de 25 años.

```
SELECT AVG(sueldo)
FROM empleado
WHERE edad > 25;
```

Se obtendría **2200** porque el promedio sólo se efectuaría entre los valores (1200, 2400, 3000)

Ejemplo 2:

La siguiente consulta calcula por cada edad el sueldo promedio, pero sólo a partir de los 25 años.

```
SELECT AVG(sueldo), edad
FROM empleado
WHERE edad > 25
GROUP BY edad;
```

Ya que primero se eliminan las tuplas con edad <= 25 y luego sobre esa relación resultante se aplica la construcción de grupos, se obtiene:

```
AVG(SUELDO)        EDAD
-----------    ----------
       2400          30
       2100          34
```

Silvia Alicia Gómez & Leticia Irene Gómez

Las funciones de agregación, como se indicó previamente, pueden también participar de la cláusula **HAVING**.

Dicha cláusula sirve para que, UNA VEZ ARMADOS LOS GRUPOS, se elimine aquellos grupos que no verifican la condición pedida. Notar que NO es equivalente al WHERE porque en lugar de eliminar tuplas antes de armar los grupos, los elimina luego de armarlos en base a UN CRITERIO GRUPAL del subconjunto en cuestión y NO un criterio individual de una tupla en particular.

Ejemplo 3:

La siguiente consulta calcula por cada edad el sueldo promedio, pero sólo a partir de los 25 años y para aquellas edades que por lo menos tengan dos representantes.

```
SELECT AVG(sueldo), edad
FROM empleado
WHERE edad > 25
GROUP BY edad
HAVING COUNT(edad) >= 2;
```

Se obtendría:

```
AVG(SUELDO)       EDAD
-----------  ----------
      2100          34
```

Para calcular esto se procede en varios pasos:

- *Primer Paso*

  Se eliminan aquellas tuplas INDIVIDUALES donde la edad sea <= 25

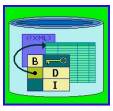
CALCULO AUXILIAR		
Nombre	**Sueldo**	**Edad**
'Juan'	1200	34
'Clara'	2400	30
'Jose'	3000	34

- *Segundo Paso*

  Se agrupa por el atributo edad formando internamente:

Calculo auxiliar		
Edad	**Sueldo**	**Nombre**
30	2400	'Clara'
34	1200	'Juan'
	3000	'Jose'

Silvia Alicia Gómez & Leticia Irene Gómez



- *Tercer Paso*

Se descartan aquellos GRUPOS (no tuplas individuales) que no tengan por lo menos dos edades. O sea desaparece el grupo formado por una tupla. Tener en cuenta que también podría usarse DISTINCT dentro de la función de agregación como se explicó previamente.

Calculo auxiliar		
Edad	**Sueldo**	**Nombre**
34	1200	'Juan'
	3000	'Jose'

- *Cuarto Paso*

Ahora se proyecta lo pedido en el SELECT. El resultado final es:

Rta	
AVG(SUELDO)	**Edad**
2100	34

2.4.7 Transformación de Expresiones

Existen distintas funciones escalares o formas de transformar un valor en otro, esto es devuelven un UNICO valor cada vez que se las invoca.

2.4.7.1 Cláusula CASE

Devuelve el resultado asociado al primer valor que satisface la expresión. Si no hay ninguna expresión verificada y no hay cláusula ELSE, entonces se devuelve NULL. Esta cláusula puede usarse en el SELECT, WHERE o HAVING.

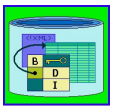
Sintaxis

```
CASE operando
WHEN valor1 THEN resultado1
WHEN valor2 THEN resultado2
...
[ELSE resultadoN]
END
```

o bien

```
CASE
WHEN expr1  THEN resultado1
WHEN expr2  THEN resultado2
...
[ELSE resultadoN]
END
```

Silvia Alicia Gómez & Leticia Irene Gómez



Ejemplo 1

Si al ejecutar la consulta

```
SELECT nota, legajo
FROM inscripto
```

se obtuviera

```
NOTA        LEGAJO
---------- ------------
          8         100
          8         200
          6         300
          5         400
          3         500
```

por medio de la cláusula CASE se podría obtener una clasificación más explícita:

```
SELECT CASE
       WHEN nota < 4 THEN 'Desaprobado'
       WHEN nota > 7 THEN 'Muy Bien'
       ELSE 'Aprobado'
       END as T, legajo
FROM inscripto
```

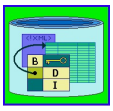
O de la otra forma:

```
SELECT CASE nota
       WHEN 1 THEN 'Desaprobado'
       WHEN 2 THEN 'Desaprobado'
       WHEN 3 THEN 'Desaprobado'
       WHEN 8 THEN 'Muy Bien'
       WHEN 9 THEN 'Muy Bien'
       WHEN 10 THEN 'Muy Bien'
       ELSE 'Aprobado'
       END as T, legajo
FROM inscripto
```

De esta forma se obtendría:

```
T                     LEGAJO
---------------------------------
Muy Bien              100
Muy Bien              200
Aprobado              300
Aprobado              400
Desaprobado           500
```

Silvia Alicia Gómez & Leticia Irene Gómez



Ejemplo 2

En este ejemplo utilizaremos dos CASE para generar dos columnas, en vez de 1.

```
SELECT legajo,
  CASE
  WHEN nota < 4 THEN 'SI'
  ELSE 'NO'
  END as Aprobado,
  CASE
  WHEN nota < 4 THEN 'NO'
  ELSE 'SI'
  END as Desaprobado
FROM inscripto
```

Con los valores del ejemplo 1, se obtendría:

```
LEGAJO           APROBADO    DESAPROBADO
----------------------------------------------
        100            NO            SI
        200            NO            SI
        300            NO            SI
        400            NO            SI
        500            SI            NO
```

2.4.7.2 Función COALESCE

Devuelve el primer valor que se evalúe como distinto de NULL.

Sintaxis

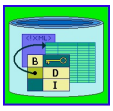
```
COALESCE( valor, valor, valor, ...)
```

En el ejercicio que se trabajó con la tabla PENALIZACIONES, aquellos jugadores que no estaban penalizados obviamente no fueron incorporados en el promedio final ya que no aparecían en la tabla PENALIZACION. Tal es el caso de los jugadores BARROS y SANCHEZ

JUGADOR		
codigo	**Nombre**	**Telefono**
A02	Barros	4542-8872
B04	Taranto	4314-2345
B06	Vilar	null
R02	Sanchez	4785-6562
R07	Peirano	4612-3332
R11	Cardoso	4314-8102

PENALIZACION		
Codigo	**Fecha**	**monto**
B06	24/10/1998	120
B04	12/05/1999	120
R07	15/07/1999	100
R07	30/10/1998	120
R11	18/07/1999	150

Silvia Alicia Gómez & Leticia Irene Gómez



Si nos hubiera interesado involucrarlos también (para que su monto afecte al promedio final), podríamos pensar modificar la consulta anidada para que tenga un cuenta un OUTER JOIN.

```
SELECT  AVG( acum)
FROM  (SELECT SUM( monto )
       FROM penalizacion RIGHT OUTER JOIN jugador
                                ON  penalizacion.codigo = jugador.codigo
        GROUP BY jugador.codigo) AS AUXI( acum )
```

Sin embargo se obtendría el mismo resultado anterior porque la tabla temporaria AUXI tendría los valores (NULL, 120, 120, NULL, 220, 150) pero sabemos que el AVG elimina los NULL antes de aplicarse.

La solución de este problema consiste en **transformar** el valor NULL en 0 (el neutro de la suma para este caso) **antes** de que se aplique la función AVG.

Existe una función de argumentos variables llamada COALESCE que devuelve de todos los argumentos el primero que se evalúa como distinto de NULL. Sólo puede devolver NULL si todos sus argumentos son NULL. Aprovechando esta función expresamos la consulta anterior:

```
SELECT  AVG( COALESCE ( acum, 0 )   )
FROM  (SELECT SUM( monto )
       FROM penalizacion RIGHT OUTER JOIN jugador
                                ON  penalizacion.codigo = jugador.codigo
        GROUP BY jugador.codigo) AS AUXI( acum )
```

La tabla temporaria que se genera es igual que la anterior pero antes de aplicar la función de agregación se transforma en (0, 120, 120, 0, 220, 150) y por lo tanto finalmente se obtiene como resultado:

```
valor
101.66666
```

2.4.7.3 Función CAST

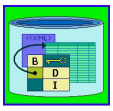
Existen DBMS que al aplicar una función, respetan los tipos originales de los argumentos. Tal es el caso de DB2, donde si el atributo MONTO fue definido de tipo INTEGER el resultado del AVG se trunca obteniéndose 101, perdiéndose precisión.

Esta función permite transformar un tipo en otro, siempre que la transformación sea posible, caso contrario se obtiene un error.

Sintaxis

```
CAST( valor  AS  nuevoTipo)
```

Silvia Alicia Gómez & Leticia Irene Gómez



Ejemplo 1:

Aplicamos la función para evitar truncamiento

```
SELECT  AVG( CAST (COALESCE ( acum, 0 ) AS DECIMAL(5,2))  )
FROM  (SELECT SUM( monto )
       FROM penalizacion RIGHT OUTER JOIN jugador
                                ON  penalizacion.codigo = jugador.codigo
        GROUP BY jugador.codigo) AS AUXI( acum )
```

O bien

```
SELECT  AVG( CAST (COALESCE ( acum, 0 ) AS FLOAT ) )
FROM  (SELECT SUM( monto )
       FROM penalizacion RIGHT OUTER JOIN jugador
                                ON  penalizacion.codigo = jugador.codigo
        GROUP BY jugador.codigo) AS AUXI( acum )
```

O bien

```
SELECT  AVG( COALESCE ( acum, 0 ) )
FROM  (SELECT CAST ( SUM( monto ) AS FLOAT )
       FROM penalizacion RIGHT OUTER JOIN jugador
                                ON  penalizacion.codigo = jugador.codigo
        GROUP BY jugador.codigo) AS AUXI( acum )
```

2.4.8 Orden Real de Evaluación de las Cláusulas

Las cláusulas son evaluadas por el DBMS en el siguiente orden (que no coincide con el orden en que deben escribirse):

1. Cláusula **FROM**
2. Cláusula **WHERE**
3. Cláusula **GROUP BY**
4. Cláusula **HAVING**
5. Cláusula **ORDER BY**
6. Cláusula **SELECT**

Silvia Alicia Gómez & Leticia Irene Gómez